Лабораторна робота №1

Потоки. Клас THREAD. Пріоритети потоків.

Виконала студентка групи МІТ-41

Пась Олександра

Хід виконання

1. Реалізуйте програму на C#, яка створює два потоки:

Перший потік виводить числа від 1 до 40 з паузою 200 мс.

```
class NumberThread
{
    public Thread Thrd;

    1 reference
    public NumberThread(string name)
    {
        Thrd = new Thread(Run);
        Thrd.Name = name;
    }

    1 reference
    public void Start()
    {
        Thrd.Start();
    }

    1 reference
    void Run()
    {
        for (int i = 1; i <= 40; i++)
        {
            Console.WriteLine(Thrd.Name + ": " + i);
            Thread.Sleep(200);
        }
        Console.WriteLine(Thrd.Name + " завершив роботу");
    }
}
```

рис. 1.1 - числовий потік

У конструкторі створюється новий об'єкт `Thread`, якому передається метод `Run()` для виконання, а також встановлюється властивість `Name` для ідентифікації потоку. Метод `Start()` слугує обгорткою для запуску потоку. Метод `Thread.Sleep(200)` призупиняє виконання потоку на 200 мілісекунд.

Другий потік виводить літери від A до Z з паузою 300 мс.

```
class LetterThread
{
    public Thread Thrd;

    1 reference
    public LetterThread(string name)
    {
        Thrd = new Thread(Run);
        Thrd.Name = name;
    }

    1 reference
    public void Start()
    {
        Thrd.Start();
    }

    1 reference
    void Run()
    {
        for (char c = 'A'; c <= 'Z'; c++)
        {
            Console.WriteLine(Thrd.Name + ": " + c);
            Thread.Sleep(300);
        }
        Console.WriteLine(Thrd.Name + " завершив роботу");
    }
}
```

рис. 1.1 - потік літер

У конструкторі ініціалізується новий потік з вказаним іменем, який виконуватиме метод `Run()`. Метод `Start()` запускає цей потік. Логіка потоку реалізована в методі `Run()`, де в циклі перебираються літери від A до Z, кожна літера виводиться на екран разом з іменем потоку, після чого потік призупиняється на 300 мілісекунд за допомогою `Thread.Sleep()`. Після завершення циклу виводиться повідомлення про завершення роботи потоку.

```
class Program
{
    0 references
    static void Main(string[] args)
    {


        // Створюємо потоки
        NumberThread nt = new NumberThread("Потік чисел");
        LetterThread lt = new LetterThread("Потік букв");

        // Запускаємо потоки (використовуємо Start())
        nt.Start();
        lt.Start();

        // Чекаємо завершення потоків
        nt.Thrd.Join();
        lt.Thrd.Join();

        Console.WriteLine("Завдання закінчено");
        Console.ReadLine();
    }
```

рис. 1.2 - program.cs

```
Потік чисел: 28
Потік чисел: 29
Потік букв: U
Потік чисел: 30
Потік букв: V
Потік чисел: 31
Потік чисел: 32
Потік букв: W
Потік чисел: 33
Потік букв: X
Потік чисел: 34
Потік чисел: 35
Потік букв: Y
Потік чисел: 36
Потік букв: Z
Потік чисел: 37
Потік чисел: 38
Потік букв завершив роботу
Потік чисел: 39
Потік чисел: 40
Потік чисел завершив роботу
Завдання закінчено
```

рис. 1.3 - результат виконання

Поясніть, як працює багатозадачність у межах одного процесу.

Багатозадачність у межах одного процесу досягається за допомогою потоків (threads). Кожен процес може містити кілька потоків, які виконуються одночасно (паралельно на різних ядрах) або почергово (на одному ядрі, перемикаючись між собою). Потоки в межах одного процесу спільно використовують його ресурси — пам'ять, відкриті файли тощо, що дозволяє їм ефективно обмінюватися даними. Операційна система або середовище виконання розподіляє час процесора між потоками, створюючи ілюзію одночасної роботи. Це дозволяє виконувати кілька завдань паралельно, наприклад, один потік може обробляти введення користувача, поки інший виконує обчислення.

2. Створіть програму, яка запускає три потоки:
Два пріоритетні (звичайні).
Один фоновий.
Організуйте так, щоб фоновий потік виконував нескінченний цикл з повідомленням у консоль.

```
0 references
class Program
{
    0 references
    static void Main(string[] args)
    {
        Console.WriteLine("Головний потік стартував");

        // Створено два пріоритетні потоки
        Thread t1 = new Thread(PriorityWork);
        t1.Name = "Пріоритетний 1";

        Thread t2 = new Thread(PriorityWork);
        t2.Name = "Пріоритетний 2";

        // Створено фоновий потік
        Thread t3 = new Thread(BackgroundWork);
        t3.Name = "Фоновий";
        t3.IsBackground = true;

        // Запускаємо всі потоки
        t1.Start();
        t2.Start();
        t3.Start();

        // Чекаємо завершення тільки пріоритетних потоків
        t1.Join();
        t2.Join();

        Console.WriteLine("Пріоритетні потоки завершились");
        Console.WriteLine("Головний потік завершується");
        Console.WriteLine("Фоновий потік зупиниться автоматично");


    }

    2 references
    static void PriorityWork()
    {
        for (int i = 1; i <= 5; i++)
        {
            Console.WriteLine(Thread.CurrentThread.Name + ": " + i);
            Thread.Sleep(500);
        }
        Console.WriteLine(Thread.CurrentThread.Name + " завершився");
    }

    1 reference
    static void BackgroundWork()
    {
        int count = 0;
        while (true)
        {
            count++;
            Console.WriteLine(Thread.CurrentThread.Name + ": " + count);
            Thread.Sleep(700);
        }
    }
```

рис. 1.4 - program.cs

У методі `Main` створюються три потоки: два звичайних (`t1` і `t2`) та один фоновий (`t3`), у якого властивість `IsBackground` встановлена в `true`. Для всіх потоків задаються імена. Потоки запускаються за допомогою методу `Start()`. Метод `Join()` викликається лише для пріоритетних потоків, що змушує головний потік чекати на їх завершення. Метод `PriorityWork` виконує цикл із 5 ітерацій з затримкою 500 мс, після чого завершується. Метод `BackgroundWork` містить нескінченний цикл, який виконується з затримкою 700 мс. Коли головний потік завершується, фоновий потік автоматично зупиняється, навіть якщо його робота не завершена. Таким чином, код ілюструє різницю між звичайними та фоновими потоками.

```
Головний потік стартував
Пріоритетний 1: 1
Пріоритетний 2: 1
Фоновий: 1
Пріоритетний 1: 2
Пріоритетний 2: 2
Фоновий: 2
Пріоритетний 2: 3
Пріоритетний 1: 3
Фоновий: 3
Пріоритетний 1: 4
Пріоритетний 2: 4
Пріоритетний 2: 5
Пріоритетний 1: 5
Фоновий: 4
Пріоритетний 1 завершився
Пріоритетний 2 завершився
Пріоритетні потоки завершились
Головний потік завершується
Фоновий потік зупиниться автоматично
```

рис. 1.5 - результат виконання завдання 2

Що відбудеться з фоновим потоком після завершення пріоритетних потоків?

Фоновий потік автоматично завершується операційною системою або середовищем виконання, коли всі передні (пріоритетні) потоки процесу завершують свою роботу. Він не блокує зупинку процесу, тому примусово переривається, навіть якщо не завершив виконання своїх задач.

3. Створити багатопотокову програму, задати різні пріоритети для потоків. Кожен потік виконує підрахунок до 100 млн. Після завершення роботи програми визначте кількість ітерацій, виконаних кожним потоком, та розрахуйте відсотковий розподіл процесорного часу.
Варіант 3) Потоків – 4 шт. (пріоритети: найнижчий, вище середнього, нижче середнього, найвищий).

```
public CountThread(string name, ThreadPriority priority, ManualResetEvent signal)
{
    _name = name;
    _priority = priority;
    _signal = signal;
    _counter = 0;
    _time = 0;
    _match = new Stopwatch();

    _thread = new Thread(Execute);
    _thread.Name = name;
    _thread.Priority = priority;
}

// Властивості для доступу до даних

public string Name => _name;

public ThreadPriority Priority => _priority;

public long Iterations => _counter;

public long ElapsedMs => _time;

public Thread Thread => _thread;

// Методи

public void Start() => _thread.Start();

public void Join() => _thread.Join();

// Скидання прапорця для нового запуску
public static void ResetRace() => _stop = false;

// Виконання

private void Execute()
{
    // Чекаємо сигнал для одночасного старту
    _signal.WaitOne();

    // Починаємо вимірювання
    _match.Reset();
    _match.Start();

    // Цикл до 100 млн або поки інший потік не завершить
    for (long i = 0; i < 100_000_000 && !_stop; i++)
    {
        _counter++;
    }

    // Зупиняємо вимірювання
    _match.Stop();
    _time = _match.ElapsedMilliseconds;

    // Якщо цей потік досяг 100 млн, зупиняємо інші
    if (_counter >= 100_000_000)
    {
        lock (_locker)
        {
            _stop = true;
        }
    }

    // Запобігаємо від ділення на нуль
    if (_time == 0) _time = 1;
}
}
```

рис. 1.6 - частина класу SimpleThread

```
static void Main(string[] args)
{
    Console.WriteLine("Дослідження пріоритетів потоків");


    var startSignal = new ManualResetEvent(false);

    // Скидаємо прапорець зупинки
    CountThread.ResetRace();

    // Створюємо 4 потоки згідно з варіантом 3
    var threads = new[]
    {
        new CountThread("Низький", ThreadPriority.Lowest, startSignal),
        new CountThread("Вище сер.", ThreadPriority.AboveNormal, startSignal),
        new CountThread("Нижче сер.", ThreadPriority.BelowNormal, startSignal),
        new CountThread("Високий", ThreadPriority.Highest, startSignal)
    };

    Console.WriteLine("Запуск потоків...\n");

    // Запускаємо всі потоки
    foreach (var t in threads)
        t.Start();

    // Даємо сигнал на одночасний старт
    startSignal.Set();

    // Чекаємо завершення всіх потоків
    foreach (var t in threads)
        t.Join();

    // Знаходимо потік-переможець (той, що досяг 100 млн)
    string winner = "";
    foreach (var t in threads)
    {
        if (t.Iterations >= 100_000_000)
        {
            winner = t.Name;
            break;
        }
    }

    Console.WriteLine($"\nПотік-переможець: {winner} (першим досяг 100 млн)");
    Console.WriteLine();

    // Розрахунок загальної кількості ітерацій
    long totalIter = 0;
    foreach (var t in threads)
        totalIter += t.Iterations;

    // Виведення результатів
    Console.WriteLine("РЕЗУЛЬТАТИ ВИКОНАННЯ:");
    Console.WriteLine("----------------------------------------------------------------------");
```

рис. 1.7 - program.cs

```
РЕЗУЛЬТАТИ ВИКОНАННЯ:
----------------------------------------------------------------------
| Потік       | Пріоритет     |      Ітерації |  Час(мс) |  CPU % |
----------------------------------------------------------------------
| Низький     | Lowest        |    45 680 988 |      383 | 15,03% |
| Вище сер.   | AboveNormal   |    98 267 677 |      393 | 32,34% |
| Нижче сер.  | BelowNormal   |    59 911 914 |      425 | 19,72% |
| Високий     | Highest       |   100 000 000 |      393 | 32,91% |
----------------------------------------------------------------------
| ВСЬОГО      |               |   303 860 579 |          |   100% |
----------------------------------------------------------------------
```

рис. 1.8 - результат виконання завдання 3

Чи завжди високопріоритетний потік отримує найбільше часу ЦП? Які фактори можуть впливати на планування?

Ні, високопріоритетний потік не завжди отримує найбільше часу ЦП. На планування впливають такі фактори: кількість ядер процесора та їх завантаження, кількість активних потоків у системі, виконання потоками операцій введення-виведення, блокування та очікування ресурсів, алгоритм планування операційної системи, пріоритети інших процесів, енергозберігаючі режими процесора та прив'язка потоків до конкретних ядер.

4. Високий рівень: Розробіть консольний застосунок, який:

- Дозволяє користувачу ввести кількість потоків та їхні пріоритети.
- Запускає кожен потік із власним завданням (наприклад, підрахунок до заданого числа).
- Виводить у реальному часі хід виконання кожного потоку (лічильник, стан).
- Після завершення – формує таблицю розподілу процесорного часу між потоками у %.

```
public class ThreadWorker
{
    4 references
    public int Id { get; }
    4 references
    public ThreadPriority Priority { get; }
    3 references
    public long Target { get; }
    6 references
    public long Count { get; private set; }
    4 references
    public long ElapsedMs { get; private set; }
    3 references
    public bool IsFinished { get; private set; }
    3 references
    public bool IsWinner { get; private set; }
    3 references
    public Thread Thread { get; }

    private readonly ManualResetEvent _startSignal;
    private static bool _stop = false;
    private static readonly object _locker = new object();

    1 reference
    public ThreadWorker(int id, ThreadPriority priority, long target, ManualResetEvent startSignal)
    {
        Id = id;
        Priority = priority;
        Target = target;
        _startSignal = startSignal;

        Thread = new Thread(Run)
        {
            Name = $"Потік {id}",
            Priority = priority
        };
    }

    1 reference
    public void Start() => Thread.Start();
    1 reference
    public void Join() => Thread.Join();
    1 reference
    public static void ResetRace() => _stop = false;

    1 reference
    private void Run()
```

рис. 1.9 - частина класу ThreadWorker

```
static void Main(string[] args)
{


    // Введення даних від користувача
    Console.Write("Введіть цільове число для підрахунку (наприклад, 10000000): ");
    long target = long.Parse(Console.ReadLine());

    Console.Write("Введіть кількість потоків: ");
    int numThreads = int.Parse(Console.ReadLine());

    var startSignal = new ManualResetEvent(false);
    var workers = new List<ThreadWorker>();

    // Скидаємо прапорець зупинки
    ThreadWorker.ResetRace();

    // Створення потоків з вибором пріоритетів
    for (int i = 1; i <= numThreads; i++)
    {
        Console.WriteLine($"\nВиберіть пріоритет для потоку {i}:");
        Console.WriteLine("1 - Lowest (Найнижчий)");
        Console.WriteLine("2 - BelowNormal (Нижче середнього)");
        Console.WriteLine("3 - Normal (Середній)");
        Console.WriteLine("4 - AboveNormal (Вище середнього)");
        Console.WriteLine("5 - Highest (Найвищий)");
        Console.Write("Ваш вибір (1-5): ");

        int choice = int.Parse(Console.ReadLine());
        ThreadPriority priority = choice switch
        {
            1 => ThreadPriority.Lowest,
            2 => ThreadPriority.BelowNormal,
            3 => ThreadPriority.Normal,
            4 => ThreadPriority.AboveNormal,
            5 => ThreadPriority.Highest,
            _ => ThreadPriority.Normal
        };

        workers.Add(new ThreadWorker(i, priority, target, startSignal));
    }

    Console.Clear();
    Console.WriteLine("=== ЗАПУСК ПОТОКІВ ===\n");
    Console.WriteLine($"Цільове число: {target:N0}");
    Console.WriteLine($"Кількість потоків: {numThreads}\n");

    // Запуск всіх потоків
    foreach (var w in workers)
    {
```

рис. 1.10 - частина program.cs

```
=== ЗАПУСК ПОТОКІВ ===

Цільове число: 2 000 000
Кількість потоків: 3

Потік 1 (Lowest) створено
Потік 2 (BelowNormal) створено
Потік 3 (Normal) створено

Натисніть Enter для одночасного старту всіх потоків...

ПОТОКИ ЗАПУЩЕНО!

Потік 1 (Lowest      ) | Статус: ЗУПИНЕНО    | Ітерацій:    1 521 171 | Прогрес:  76,06%
Потік 2 (BelowNormal ) | Статус: ПЕРЕМОЖЕЦЬ  | Ітерацій:    2 000 000 | Прогрес: 100,00%
Потік 3 (Normal      ) | Статус: ЗУПИНЕНО    | Ітерацій:    1 891 533 | Прогрес:  94,58%


=== РЕЗУЛЬТАТИ ВИКОНАННЯ ===
--------------------------------------------------------------------------------
Потік    Пріоритет              Ітерації    Час(мс)    CPU % Статус
--------------------------------------------------------------------------------
Потік 1    Lowest                1 521 171         20   28,10% ЗУПИНЕНО
Потік 2    BelowNormal           2 000 000         20   36,95% ПЕРЕМОЖЕЦЬ
Потік 3    Normal                1 891 533         20   34,95% ЗУПИНЕНО
--------------------------------------------------------------------------------
ВСЬОГО                         5 412 704               100%
--------------------------------------------------------------------------------

Натисніть Enter для виходу...
```

рис. 1.11 - результат виконання завдання 4

Висновок: У ході виконання лабораторної роботи було досліджено принципи багатопотокового програмування в C#. Експериментально підтверджено, що пріоритети потоків впливають на розподіл процесорного часу: потоки з вищим пріоритетом отримують більше часу ЦП і виконуються швидше. Встановлено, що фоновий потік автоматично завершується після зупинки всіх пріоритетних потоків. Реалізовано програму зі змаганням потоків, яка дозволяє спостерігати за виконанням у реальному часі та аналізувати відсотковий розподіл процесорного часу між потоками з різними пріоритетами. Отримані результати

демонструють ефективність використання багатопотоковості для паралельного виконання завдань.